

Verified Compilers

An Annotated Bibliography

Amirali Ebrahimzadeh (amiralie@umich.edu)
University of Michigan

Introduction

Even verified programs can be compiled to unsafe, buggy code if compiled using unverified compilers. Using verified compilers is essential for ensuring safety of critical software. The core property of interest in verified compilation is semantic preservation, which ensures that a source program and its compiled code have the exact same observational behavior, e.g. they must produce the exact same I/O trace. A verified compiler conforms to this specification, where as a buggy compiler can compile a source code to machine code with different behaviour.

Half a century after the first paper on verifying compilers [5], this field has grown drastically, as many practical tools and verification paradigms have been developed for compiler safety.

Evolution of Verified Compilation

The earliest effort toward this goal was the first proof of compiler correctness by McCarthy and Painter in 1967 [5]. They provided a handwritten proof of semantic preservation for a simple language of arithmetic expressions. While it served as a vital proof of concept, the language was a toy language, and the proof was not checked mechanically.

A major breakthrough occurred with CompCert, developed by Xavier Leroy in 2009 [2]. CompCert is a realistic compiler that translates a subset of C (CLight) to PowerPC architecture, which was popular in the safety critical avionics industry. It utilizes the Coq proof assistant for both its implementation and proof across 14 intermediate languages. While it achieved high performance—running only 7% to 12% slower than GCC optimization

levels O1 and O2—it left components like the parser and assembler completely unverified.

To address the limitations of unverified frontends, CakeML (Kumar et al., 2014) introduced end-to-end verified compilation [1]. It translates a subset of Standard ML to x86-64 machine code, with its verification entirely mechanized within the HOL4 interactive theorem prover. CakeML achieved a verified front end, including lexing, parsing, and type checking, along with verified garbage collection. This provides greater confidence in the correctness of a larger portion of the pipeline, unlike the initial versions of CompCert that mostly verified the middle stages of compilation. Furthermore, CakeML was written in CakeML itself and was evaluated to x86 assembly by itself. This bootstrapping property is a core contribution of the paper, as the compiler is itself compiled by a verified compiler, yielding safety for the executable binary.

Previous mentioned work consist of developing new verified compilers to replace the existing unverified ones. On the other hand, a newer thread of research focuses on bringing verification to valuable existing industrial compilers. Transitioning to industrial infrastructures, VeLLVM (Zhao et al., 2012) formalizes the LLVM Intermediate Representation (IR) in Coq [8]. VeLLVM uses translation validation, providing a verified checker that runs after unverified LLVM optimizations to prove semantic equivalence.

Alive (Lopes et al., 2015) brings automation to the tedious process of compiler verification by introduced SMT-based verification specifically for peephole optimizations [4]. Using a domain-specific language equipped with an SMT solver, it translated and automatically verified the complex InstCombine pass in LLVM. Alive successfully discovered 8 bugs in the LLVM framework while introducing zero performance overhead to the compiled binaries.

Verifying a compiler is an expensive task. An alternative and cheaper paradigm is Type Preserving Compilation, demonstrated by Typed Assembly Language (TAL) (Morrisett et al., 1999) [6]. Rather than verifying the compiler itself, TAL pushes the type system from the source language (System F) all the way down to the assembly level. By running a type-checker before the host machine runs the code, TAL guarantees memory and control-flow safety, safely permitting the execution of untrusted code without needing to trust the compiler.

Current Status and Open Questions

Today, verified compilation has transitioned from theoretical proofs of concept to highly practical, industrial-strength infrastructure. Compilers like CompCert and CakeML have demonstrated that complete semantic preservation is practically viable, with CompCert seeing active adoption in safety-critical industries like avionics, and supporting a larger subset of C covering almost all of its core features. Furthermore CompCert can now generate code for a wide variety of target architectures, like ARM, PowerPC, RISC-V and x86.

Several major open research questions remain in the field. Two prominent challenges include achieving compositional compiler correctness [7]—developing frameworks that guarantee safety when linking separately compiled modules and interacting with unverified foreign code—and scaling automated translation validation. This latter area focuses on using SMT solvers to automatically certify optimization passes within existing industrial infrastructures (such as LLVM and MLIR) [3], bringing formal verification to mainstream compilers without the massive manual proof effort required to build them from scratch. Ultimately, the open question is how to integrate practical verification methods into modern, complex compiler pipelines with minimal overhead and engineering effort.

References

- [1] R. Kumar, M. O. Myreen, M. Norrish, and S. Owens, “Cakeml: A verified implementation of ml,” in *Proceedings of the 41st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL ’14)*. ACM, 2014, pp. 179–191.

The paper confronts the problem that unverified compilers constitute a large, complex part of the trusted computing base for program verification. Furthermore, previous verified compilers for general-purpose languages have failed to address both the compilation algorithm and the execution of that algorithm in machine code end-to-end. As a solution, the authors developed CakeML, a mechanically verified implementation of a substantial subset of Standard ML. It is implemented as an interactive read-eval-print loop (REPL) in x86-64 machine code. The compiler is verified within the HOL4 theorem prover and is capable of bootstrapping itself, applying the verified com-

piler to its own source code to produce verified machine code. The authors evaluate their solution through an end-to-end correctness theorem ensuring the REPL prints only results permitted by the language semantics, and by running small benchmarks (e.g., Fibonacci, Quicksort) against interpreted OCaml and ML. For instance, CakeML ran the Fibonacci benchmark 2.2 times faster than interpreted OCaml. Limitations include the lack of a verified completeness proof for the type inferencer, restrictions on polymorphism for let-bound variables, and a reliance on the target machine code semantics accurately reflecting actual hardware processor behavior. Future work aims to extend the system to generate ARM machine code, add source-level optimizations, and deploy CakeML as an execution environment for a theorem prover.

- [2] X. Leroy, “Formal verification of a realistic compiler,” *Communications of the ACM*, vol. 52, no. 7, pp. 107–115, 2009.

The paper confronts the problem that compilers are complex programs that can introduce bugs, silently generating incorrect executables for correct source programs. This is particularly dangerous for safety-critical software, where formal verification of the source code can be entirely invalidated by an untrustworthy compiler. As a solution, the paper presents CompCert, a realistic, verified compiler that translates Clight (a large subset of C) into PowerPC assembly code. The compiler and its end-to-end proof of semantic preservation are mechanically verified using the Coq proof assistant, processing the code through 14 passes and eight intermediate languages. The author evaluates the compiler’s performance by benchmarking the generated code against GCC 4.0.1 on a PowerPC processor using a custom suite of programs. The evaluation demonstrates that CompCert generates code more than twice as fast as GCC without optimizations, and on average is only 7% slower than GCC at optimization level 1, and 12% slower at level 2. Limitations of the system include its omission of certain C features like the ‘goto’ statement and extended-precision arithmetic, as well as its reliance on unverified upstream and downstream components, namely the CIL-based parser, the assembler, and the linker. Future work aims to handle a larger subset of C,

deploy additional optimizations, target other processors, and extend the proofs to handle shared-memory concurrency.

- [3] N. P. Lopes, J. Lee, C.-K. Hur, Z. Liu, and J. Regehr, “Alive2: Bounded translation validation for llvm,” in *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*. ACM, 2021, pp. 65–79.

The paper confronts the problem that scaling automated translation validation to a massive, evolving industrial compiler like LLVM is exceptionally difficult due to complex memory models, inter-procedural optimizations, and the size of the codebase. As a solution, the authors present Alive2, a bounded translation validation tool that formally models the LLVM intermediate representation and uses SMT solvers to automatically verify the correctness of LLVM optimization passes without requiring manual proofs. The authors evaluated Alive2 by running it over the LLVM unit test suite and the SPEC CPU benchmarks, successfully identifying dozens of new, previously unknown bugs in LLVM’s optimization pipeline. Limitations include the tool’s reliance on bounded verification (which cannot guarantee the absence of all bugs) and SMT solver timeouts on highly complex loops.

- [4] N. P. Lopes, D. Menendez, S. Nagarakatte, and J. Regehr, “Provably correct peephole optimizations with alive,” in *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation*. ACM, 2015, pp. 22–32.

The paper confronts the problem that compiler optimizations—specifically local peephole optimizations—are notoriously difficult to implement correctly and represent a persistent source of subtle bugs, largely due to the complexities of reasoning about boundary conditions and undefined behaviors. As a solution, the authors present Alive, a domain-specific language designed for specifying peephole optimizations for the LLVM intermediate representation. Alive utilizes SMT solvers to automatically prove the correctness of these transformations (accurately modeling LLVM’s three types of undefined behavior) or generate counterexamples, and it can automatically translate the verified optimizations into C++

code suitable for a compiler pass. The authors evaluated Alive by translating 334 existing LLVM InstCombine optimizations, discovering 8 previously unknown bugs in the LLVM framework. Furthermore, they compiled the SPEC 2000 and 2006 benchmarks (approximately one million lines of code) using an LLVM compiler where the InstCombine pass was replaced by Alive-generated C++ code, doing so with zero unexpected test failures. Limitations of the system include its inability to currently handle floating-point types, vectors, and aggregate types, as well as performance bottlenecks where verification can take hours for larger bitwidths involving multiplication and division. Future work aims to replace a larger portion of the LLVM InstCombine pass with Alive-generated code, implement optimizations in the C++ code generator (such as reusing redundant pattern-match calls), and improve the SMT solver querying process by biasing solver results.

- [5] J. McCarthy and J. Painter, “Correctness of a compiler for arithmetic expressions,” *Mathematical aspects of computer science*, vol. 19, p. 33, 1967.

The paper confronts the issue that compilers are typically trusted without formal proof, aiming to establish a prototype for proving the correctness of usable compilers so that computers can eventually check these proofs. As a solution, McCarthy and Painter provide a formal definition of correctness and a compiling algorithm that translates arithmetic expressions (consisting of constants, variables, and binary sums) into the machine language of a single-address computer with an accumulator. They evaluate their solution by providing a handwritten mathematical proof, utilizing an induction on the abstract syntax of the expressions being compiled to guarantee semantic preservation. A major limitation of this early work is its reliance on severely restricted toy languages; for instance, the object language has no jump instructions. The authors note that future work must address extensions like variable length sums, assignment statements, conditional expressions, and “go to” statements, which will require a complete revision of their formalism.

- [6] G. Morrisett, D. Walker, K. Crary, and N. Glew, “From system f to typed assembly language,” *ACM Transactions on Programming Languages and Systems (TOPLAS)*, vol. 21, no. 3, pp. 527–568, 1999.

The paper confronts the problem that while many optimizing compilers use typed intermediate languages, there is a conceptual line where types are lost in the final stages of compilation. Because the final assembly code loses its type information, we cannot easily check if it obeys basic safety rules, making it dangerous to run code from unknown sources. As a solution, Morrisett et al. introduce a Typed Assembly Language (TAL) based on a conventional RISC instruction set. They demonstrate the expressiveness of TAL by specifying a type-preserving compiler that translates a high-level language (System F) to TAL through a sequence of intermediate transformations, including CPS conversion, closure conversion, hoisting, allocation, and code generation. The authors evaluate their solution theoretically by proving the type safety of TAL via Subject Reduction and Progress theorems, guaranteeing that well-formed programs never become “stuck”. They also demonstrate its practical applicability by implementing TALx86 for the Intel 32-bit Architecture and building compilers for a safe C-like language and a subset of Scheme. A limitation of the presented simple TAL is that it does not support advanced implementation techniques like run-time code generation, intensional polymorphism, and array bounds check elimination. Future work aims to introduce dependent types to eliminate array bounds checks, extend the language to allow run-time type analysis for data-layout optimizations, construct robust proofs of correctness for translations involving recursive types or objects, and extend the type system to the generality of Proof-Carrying Code.

- [7] G. Stewart, L. Beringer, S. Cuellar, and A. W. Appel, “Compositional compcert,” in *Proceedings of the 42nd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. ACM, 2015, pp. 275–287.

The paper confronts the problem that the original CompCert only provides whole-program verification, meaning its formal safety guarantees are voided if a compiled program is linked

with modules compiled separately or written in different languages. As a solution, the authors develop Compositional CompCert, which refines the compiler’s semantics to support separate compilation and linking by using a novel structured simulation proof technique. They evaluate their solution by successfully linking CompCert-compiled C modules with unverified modules and proving end-to-end contextual equivalence. Future work points toward integrating these compositional proofs with concurrent memory models and more complex foreign function interfaces.

- [8] J. Zhao, S. Nagarakatte, M. M. K. Martin, and S. Zdancewic, “Formalizing the llvm intermediate representation for verified program transformations,” in *Proceedings of the 39th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. ACM, 2012.

The paper confronts the problem of how to bring formal safety to LLVM, a massive and highly optimized C++ compiler infrastructure, without abandoning existing compilers. A major challenge in this space is that the LLVM Intermediate Representation (IR)—a strictly typed, RISC-like, Static Single Assignment language—is notoriously complex and features intentional undefined behaviors. As a solution, Zhao et al. introduce VeLLVM, which provides a formal semantics of the LLVM IR within the Coq proof assistant. To ensure correctness, VeLLVM utilizes translation validation by providing a Coq-verified checker that runs after an unverified LLVM optimization. This checker proves that the optimized program is semantically equivalent to the unoptimized program after each step. The authors evaluate VeLLVM by extracting an executable interpreter from their Coq formalization and successfully running 134 out of 145 tests from the LLVM regression test suite. Furthermore, they demonstrate the framework’s practicality by verifying and extracting a memory-safety transformation pass called SoftBound, which successfully detected all buffer overflows in the NIST Juliet test suite and operated with similar performance overhead to the original unverified C++ implementation. Limitations of the current formalization include a lack of support for exception handling, vector types, and certain intrinsic functions like variable arguments. Future work

aims to build upon this formal semantics to eventually construct a fully verified LLVM compiler analogous to CompCert.

CSE-590 Project Implementation Proposal

Amirali Ebrahimzadeh

Introduction

While traditional compiler verification uses formal methods to prove the correctness of existing passes [5], this project explores an inverted paradigm: using formal methods as an *on-demand enabler* for aggressive optimizations that standard compilers abandon due to conservatism. Given the project scope, the implementation will focus on a proof-of-concept latency-bounded oracle integrated into LLVM.

Architecture: The On-Demand Safety Oracle

Modern production compilers rely on lightweight, fast analyses (e.g., LLVM’s `ScalarEvolution` or `ValueTracking`). When these fail to prove the safety of a transformation, the optimization is aborted. The proposed architecture introduces a modular, multi-backend oracle that is invoked strictly when these fast passes fail.

The architecture consists of three components:

1. **The Trigger:** A modified optimization pass (e.g., Loop Invariant Code Motion, as discussed in Scott Mahlke’s EECS 583 lectures [1]) that identifies high-value candidates that fail standard checks.
2. **The Post-Condition Deriver:** An off-the-shelf Abstract Interpretation engine (such as Crab-LLVM [2]) or an SMT-based engine (like SeaHorn [3]). This engine computes the program invariants, yielding a mathematical post-condition (ϕ_{post}) at the optimization site.
3. **The Verification Query:** A fast implication check mapping the derived state to the optimization’s required precondition (ϕ_{safety}). If $\phi_{post} \implies \phi_{safety}$ holds true via lattice inclusion or a lightweight Z3 SMT query, the optimization is forcefully enabled.

Motivating Example: Relational Transitivity and Dead Code

Consider the following logical puzzle reliant on relational transitivity:

```
void dead_code_elim(int a, int b, int c, int j) {
    int i = 0;

    if (a > b)
        if (b > c)
            if (c > a)
                i = j / 15;

    printf("%d", i);
}
```

Based on my testing, standard LLVM (even at `-O3`) fails to identify the mathematical contradiction. Instead of eliminating the unreachable code, LLVM generates instructions to evaluate the conditions at runtime and uses speculative execution (`select`) for the division. In our framework:

- The oracle extracts the relational conditions leading to the division branch.
- A lightweight SMT query (e.g., via Z3) checks the satisfiability of the conjunction: $a > b \wedge b > c \wedge c > a$.
- The solver instantly returns **UNSAT** (unsatisfiable) due to the transitivity contradiction.
- The compiler pass leverages this proof to perform Advanced Dead Code Elimination (DCE), safely deleting the entire branch and hardcoding the output to 0.

Extensibility and Other Applications

This modular architecture—abstracting the math engine behind a `SafetyOracle` API—allows developers to explicitly toggle the compile-time latency vs. aggressiveness trade-off. Beyond relational dead code elimination, these constraints can be formulated for:

- **Safe Instruction Hoisting:** Proving loop-invariants (e.g., a denominator $i + 1 \neq 0$) to safely hoist instructions like scalar division out of complex loops.
- **Memory Disambiguation:** Proving memory locations `r1` and `r2` are strictly unequal or valid memory locations, enabling aggressive load/store hoisting and optimal instruction scheduling.
- **Superoptimization Parallels:** Unlike Souper [4], which acts as an offline synthesizer for peephole patterns, this approach functions as an online, control-flow-aware verifier.

All the mentioned candidate optimizations can be reduced to integer and real arithmetic formulas.

Oracle Backend Evaluation: Expressiveness vs. Latency

The viability of an on-demand safety oracle depends entirely on the mathematical backend used to derive the post-conditions. Because formal verification suffers from exponential time complexity on complex loops, our architecture is explicitly modular, allowing the developer to toggle backends based on their compile-time budget. Note that all of the evaluated engines natively accept LLVM Intermediate Representation (IR) directly as input. Table 1 outlines the candidate engines.

Engine	Query Fashion	Mechanism	Expressiveness	Latency
Crab (Intervals)	API (Lattice Inclusion)	Abstract Interp.	Low (Non-relational)	Milliseconds
Crab (Octagons)	API (Lattice Inclusion)	Abstract Interp.	Medium (Linear relations)	Seconds
IKOS	API (Invariant State)	Abstract Interp.	High (Memory/Pointers)	Sec to Mins
SeaHorn	SMT (Satisfiability)	SMT / Horn Clauses	Maximum (Non-linear)	Mins / Timeouts

Table 1: Comparison of post-condition derivation engines.

While SeaHorn provides the maximum mathematical rigor required for complex non-linear invariants, its SMT-based search space often exceeds standard compilation time budgets. Conversely,

Abstract Interpretation engines like Crab-LLVM, specifically when using the Octagon domain, provide the optimal balance for this project. They compute sufficiently strong relational invariants algebraically, safely staying within a strict sub-5-second compilation budget.

References

- [1] Scott Mahlke, *EECS 583: Advanced Compilers*, University of Michigan. <https://web.eecs.umich.edu/~mahlke/courses/583f25/lectures.html>
- [2] Jorge A. Navas et al., *Crab: A C++ Library for Building Program Static Analyzers*.
- [3] Arie Gurfinkel et al., *The SeaHorn Verification Framework*.
- [4] Sasnauskas et al., *Souper: A Synthesizing Superoptimizer*, 2017.
- [5] Lopes et al., *Provably Correct Peephole Optimizations with Alive*, PLDI 2015.